

Report Cyber Security MaintAI MVP Demo

Data audit: 2026-05-26

Repository: E:\PROGETTI\MAINTAI_MVP_DEMO

Versione dichiarata: MaintAI 3.1.7 / frontend package 3.1.7

Tipo audit: revisione statica del codice, configurazione, dipendenze e superfici di attacco principali. Non e' stato eseguito un penetration test contro produzione.

Executive Summary

Il progetto ha gia' alcuni controlli importanti: JWT con secret obbligatorio, cifratura Fernet obbligatoria per password IMAP, bcrypt per password utente, cookie HttpOnly, CSRF origin-check sulle richieste mutanti, login rate-limited e filtri tenant_id diffusi.

I rischi piu' importanti oggi sono pero' concentrati in quattro aree:

- 1 Autorizzazione applicativa insufficiente: molti endpoint critici richiedono solo autenticazione/tenant, non ruolo. Un tecnico autenticato potrebbe modificare asset, tecnici, pianificazioni e piani.
- 2 Dipendenze vulnerabili: next@16.1.6 ha advisory alte; python-multipart==0.0.22 espone DoS su upload; cryptography==46.0.6 ha CVE corretta in 46.0.7.
- 3 Upload e file pubblici: allegati e manuali sono accettati con controlli parziali, caricati in memoria e pubblicati via /uploads o URL pubblici Supabase.
- 4 Superfici pubbliche e desktop: checklist pubblica senza rate limit/expiry, Tauri con CSP disabilitata e permesso shell troppo ampio.

Priorita consigliata:

- Entro 24 ore: aggiornare dipendenze vulnerabili, introdurre RBAC sugli endpoint mutanti, limitare upload a livello proxy/app, mettere rate limit agli endpoint pubblici e AI.
- Entro 7 giorni: rendere privati gli allegati, aggiungere security headers/CSP, ridurre privilegi Tauri, harden CORS e cookie in produzione.
- Entro 30 giorni: threat model multi-tenant, test automatici RBAC/IDOR, secret scanning CI, audit trail immutabile e policy AI/data governance.

Metodologia

Comandi e controlli eseguiti:

- Ricerca statica con rg su auth, JWT, CORS, CSRF, upload, file, tenant, localStorage, Tauri, logging.
- Revisione mirata di backend/core/security.py, backend/main.py, route FastAPI principali, frontend/app/lib/api.ts, proxy Next e configurazione Tauri.
- npm audit --json e npm audit --omit=dev --json in frontend.
- pip-audit -r backend/requirements.txt -f json nella .env.
- cargo tree -e no-dev; cargo audit non era installato, quindi non ho un riscontro CVE Rust completo.
- Secret scan locale redatto: non ho riportato valori sensibili nel report.

Scala severita:

- Critica: compromissione plausibile di tenant, privilegi, segreti o integrita' operativa.
- Alta: sfruttamento realistico con impatto importante su confidenzialita', integrita' o disponibilita'.
- Media: rischio significativo ma con prerequisiti, impatto limitato o mitigazioni parziali.
- Bassa: hardening, hygiene o difesa in profondita'.

Punti Forti Osservati

- JWT_SECRET e ENCRYPTION_KEY sono obbligatorie all'avvio: backend/core/security.py:18 e backend/core/security.py:42.
- Password utente con bcrypt: backend/core/security.py:106.
- Cookie auth HttpOnly: backend/api/routes/auth.py:63.
- Logout con blacklist jti e invalidazione su cambio password tramite token_version: backend/api/routes/auth.py:117, backend/core/security.py:186, backend/core/security.py:196.
- Rate limit login: backend/api/routes/auth.py:23.
- CSRF fail-closed per richieste mutanti senza Bearer: backend/main.py:692.
- Isolamento tenant spesso presente in query e repository.

Finding C-01 - RBAC insufficiente sugli endpoint mutanti

Severita: Critica **Categoria: Broken Access Control / OWASP A01** **Evidenza:**

- Asset mutabili con solo get_current_tenant_id: backend/api/routes/assets.py:68, backend/api/routes/assets.py:78, backend/api/routes/assets.py:86.
- Tecnici e assenze mutabili con solo get_current_tenant_id: backend/api/routes/tecnici.py:139, backend/api/routes/tecnici.py:148, backend/api/routes/tecnici.py:177, backend/api/routes/tecnici.py:195, backend/api/routes/tecnici.py:215.
- Planning mutabile/generabile con solo tenant su molti endpoint: backend/api/routes/planning.py:290, backend/api/routes/planning.py:431, backend/api/routes/planning.py:1122, backend/api/routes/planning.py:1486.
- Piani manutenzione mutabili con solo tenant: backend/api/routes/piano_manutenzione.py:228, backend/api/routes/piano_manutenzione.py:315, backend/api/routes/piano_manutenzione.py:353, backend/api/routes/piano_manutenzione.py:443.
- Email config creabile/cancellabile con solo tenant: backend/api/routes/email_config.py:74, backend/api/routes/email_config.py:102.

Motivazione:

get_current_tenant_id autentica l'utente e risolve il tenant, ma non decide cosa l'utente puo' fare. Nel modello del progetto esistono ruoli distinti (superadmin, responsabile, tecnico), e i tecnici target dovrebbero eseguire ticket, non amministrare anagrafiche, tecnici, piani o configurazioni email. Senza RBAC, un token valido a basso privilegio diventa sufficiente per modificare dati critici del tenant.

Scenario realistico:

Un tecnico autenticato, o un attaccante con il token di un tecnico, invia DELETE /assets/{id}, POST /planning/clear, PUT /tecnici/{id} o crea una configurazione IMAP. L'impatto e' operativo: perdita integrita' piano, cancellazione logica dati, alterazione turni/assenze, abuso costi AI.

Soluzioni:

1 Creare dipendenze centrali:

```
def require_roles(*roles: str):
    def dep(payload: dict = Depends(get_current_user_payload)):
        if payload.get("ruolo") not in roles:
            raise HTTPException(status_code=403, detail="Permessi insufficienti")
        return payload
    return dep
```

1 Applicare policy per dominio:

- Asset/impianti/siti/tecnici/assenze/piani/email config/planning generate-confirm-clear: responsabile o superadmin.
- - Ticket create/update esecuzione: tecnico, responsabile, superadmin, ma con regole: tecnico puo' aggiornare solo ticket assegnati o azioni consentite.
- - Logs, tenant, bulk import: solo superadmin.
- - Checklist pubblica: senza auth ma con token limitato e anti-spam.

1 Aggiungere test automatici:

- tecnico non puo' creare asset.
- - tecnico non puo' cancellare piano.
- - tecnico puo' chiudere solo ticket assegnato.
- - responsabile puo' gestire risorse del proprio tenant.
- - superadmin senza X-Tenant-Id non deve mutare dati tenant-specifici salvo endpoint globali.

1 Documentare una matrice endpoint -> ruolo in docs/security.md o OpenAPI.

Finding C-02 - Dipendenze frontend vulnerabili, soprattutto Next.js

Severita: Critica/Alta**Categoria: Supply Chain / Known Vulnerable Components****Evidenza:**

- frontend/package.json: 25 usa next: 16.1.6.
- npm audit --omit=dev segnala 10 vulnerabilita' totali: 3 high e 7 moderate.
- next ha fix disponibile a 16.2.6, con advisory su DoS, SSRF, middleware/proxy bypass, request smuggling, cache poisoning e XSS in casi specifici.
- Altri pacchetti transitivi vulnerabili: fast-uri, picomatch, hono, @hono/node-server, postcss, qs, brace-expansion, ip-address.

Motivazione:

Next e' parte del perimetro esposto su Vercel. Anche se non tutti gli advisory sono sfruttabili nella configurazione attuale, la presenza di proxy App Router (frontend/app/api/[...path]/route.ts) e server components rende gli

advisory Next ad alta priorita: bypass del middleware/proxy e request smuggling possono invalidare assunzioni di auth, caching e routing.

Soluzioni:

1 Aggiornare almeno:

```
cd frontend
npm install next@16.2.6 eslint-config-next@16.2.6
npm audit fix
npm run build
npm run lint
```

- 1 Valutare una versione Next piu' recente se stabile nel progetto.
- 2 Tenere package-lock.json aggiornato e committato.
- 3 Aggiungere CI con `npm audit --omit=dev --audit-level=high`.
- 4 Dopo upgrade, testare proxy `/api/[...path]`, login, cookie, static export desktop e Vercel build.

Finding C-03 - Segreti/config sensibili in file locali tracciati o facilmente distribuibili

Severita: Critica se i valori sono reali; Media se sono solo placeholder **Categoria:** Secret Management **Evidenza:**

- `.claude/settings.local.json` e file in `.claude/` risultano tracciati da git.
- La scansione redatta ha trovato riferimenti a `DATABASE_URL`, `JWT_SECRET`, `TAURI_KEY_PASSWORD` in `.claude/settings.local.json`.
- `tauri_signing_key.txt`, `backend/.env`, `maintai.db` sono ignorati correttamente, ma esistono nel workspace locale.

Motivazione:

I file "local settings" spesso contengono comandi, token temporanei o variabili d'ambiente reali. Se sono tracciati e pushati, qualunque clone del repository puo' recuperare segreti o informazioni operative. Per JWT/Fernet/Tauri signing, la conseguenza e' grave: forgiare sessioni, decifrare password IMAP o firmare update desktop malevoli.

Soluzioni:

- 1 Verificare subito il contenuto reale di `.claude/settings.local.json` senza condividerlo.
- 2 Se contiene valori reali, ruotare:

- `JWT_SECRET`
- `- ENCRYPTION_KEY`
- `- credenziali DB/Supabase/OpenAI` eventualmente presenti
- `- password o chiave privata signing Tauri`

1 Rimuovere dal tracking:

```
git rm --cached .claude/settings.local.json
```

1 Aggiungere a .gitignore:

```
.claude/settings.local.json
.claude/worktrees/
.codex/local*
frontend/out/
```

- 1 Se valori reali sono già finiti nella history remota, usare git filter-repo o BFG e considerare comunque i segreti compromessi.
- 2 Aggiungere secret scanning in CI: Gitleaks o TruffleHog.

Finding H-01 - Checklist pubblica senza scadenza/rate limit e con creazione ticket anonima

Severita: Alta**Categoria: Abuse / Public Endpoint Exposure****Evidenza:**

- Token pubblico generato con UUID: backend/api/routes/check_primo_livello.py:96.
- Lettura pubblica senza auth: backend/api/routes/check_primo_livello.py:110.
- Creazione ticket pubblica senza auth: backend/api/routes/check_primo_livello.py:138.
- L'endpoint pubblico restituisce anche nome asset e token: backend/api/routes/check_primo_livello.py:36, backend/api/routes/check_primo_livello.py:42, backend/api/routes/check_primo_livello.py:128.

Motivazione:

Il QR pubblico è funzionale, ma un token QR fotografato, condiviso o indicizzato permette creazione illimitata di ticket BD. Non c'è scadenza, revoca, throttling, captcha, limite giornaliero o approvazione. Questo può causare spam operativo e saturare planner/dashboard.

Soluzioni:

- 1 Aggiungere campi public_token_expires_at, public_token_revoked_at, last_public_submission_at.
- 2 Rate limit dedicato: per IP + token, es. 5/minute, 20/day.
- 3 Limitare lunghezza input e normalizzare:
 - descrizione: max 2000 char.
 - - operatore: max 80 char.
- 1 Valutare CAPTCHA leggero o codice giornaliero per endpoint anonimi.
- 2 Non restituire public_token nella response pubblica.
- 3 Rendere la creazione ticket una "segnalazione" da approvare se il contesto richiede controllo.
- 4 Loggare fingerprint IP/user-agent in modo minimizzato.

Finding H-02 - Upload allegati/manuali esposti pubblicamente e validati solo parzialmente

Severita: Alta**Categoria: Unrestricted File Upload / Sensitive File Exposure****Evidenza:**

- Local fallback monta /uploads come statico pubblico: backend/main.py:785.
- Supabase storage restituisce URL pubblico: backend/core/storage.py:37.
- Local storage restituisce /uploads/<filename>: backend/core/storage.py:43.
- Allegati ticket accettano estensioni ampie, inclusi .zip, video, Office: backend/api/routes/tickets.py:28.
- Validazione allegato basata su estensione: backend/api/routes/tickets.py:540.
- Manuali PDF letti e parsati da input utente: backend/api/routes/manuali.py:49, backend/api/routes/manuali.py:58.

Motivazione:

Gli allegati di ticket industriali possono contenere dati tecnici, foto impianti, firme, documentazione o informazioni personali. Se sono pubblici per URL, chiunque ottenga il link puo' scaricarli. Inoltre validare solo estensione non impedisce file camuffati, malware in zip/office, payload HTML serviti con content-type ambiguo o file molto costosi da parsare.

Soluzioni:

1 Rendere gli allegati privati:

- Supabase bucket privato.
- - URL firmati con scadenza breve.
- - Endpoint download autenticato che verifica tenant e ticket.

1 Non montare /uploads direttamente in produzione o ambienti condivisi.

2 Forzare download con Content-Disposition: attachment e X-Content-Type-Options: nosniff.

3 Validare magic bytes/MIME reale, non solo estensione.

4 Rimuovere .zip salvo reale necessita'; se resta, limitarne dimensione e contenuto.

5 Antivirus/scanner asincrono per Office/zip/pdf.

6 Limiti a livello proxy/server: body max, request timeout, numero file, dimensione multipart.

7 Stream su disco invece di await file.read() per file grandi.

Finding H-03 - DoS su multipart/upload e parsing file

Severita: Alta **Categoria:** Denial of Service **Evidenza:**

- python-multipart==0.0.22: backend/requirements.txt:21.
- pip-audit segnala:
- CVE-2026-40347, fix 0.0.26.
- CVE-2026-42561, fix 0.0.27.
- Upload multipli usano await file.read() in memoria: manuali backend/api/routes/manuali.py:49, allegati backend/api/routes/tickets.py:547, bulk import backend/api/routes/bulk_import.py:386 e backend/api/routes/bulk_import.py:486, piani backend/api/routes/piano_manutenzione.py:744 e backend/api/routes/piano_manutenzione.py:814.
- Bulk import non mostra un limite esplicito di dimensione file.

Motivazione:

Le CVE di python-multipart permettono consumo CPU con multipart malformati. Anche senza CVE, leggere interamente file in memoria consente a pochi upload concorrenti di saturare worker e RAM. PDF/Excel parsing aggiunge carico CPU ulteriore.

Soluzioni:

1 Aggiornare:

```
pip install "python-multipart>=0.0.27" "cryptography>=46.0.7" "pytest>=9.0.3"
pip freeze | rg "python-multipart|cryptography|pytest"
```

1 Configurare limiti edge:

- Vercel/Render/proxy: max body.
- - Uvicorn/Gunicorn worker limits e timeout.

- 1 Implementare dimension limit prima del parsing; per upload con Content-Length assente, abort dopo N byte.
- 2 Per Excel/bulk import: limite es. 5 MB e massimo righe.
- 3 Per PDF: limite pagine e tempo di parsing.
- 4 Coda asincrona per parsing manuali invece di bloccare richiesta web.

Finding H-04 - Gestione token/cookie migliorabile

Severita: Alta**Categoria: Session Management****Evidenza:**

- Token JWT 7 giorni: backend/core/security.py:34.
- COOKIE_SECURE default false: backend/core/security.py:77.
- COOKIE_SAMESITE configurabile senza enforcement produzione: backend/core/security.py:78.
- Login restituisce sempre access_token nel JSON: backend/api/routes/auth.py:78.
- Frontend salva token Tauri in localStorage: frontend/app/lib/api.ts:12, frontend/app/lib/api.ts:16.

Motivazione:

HttpOnly protegge il cookie dal furto via XSS, ma restituire il JWT nel body lo rende disponibile al codice JS dopo login. Nel desktop Tauri il token finisce in localStorage; in caso di XSS o WebView compromise, e' esfiltrabile. Il default COOKIE_SECURE=false e' sicuro solo in locale: se misconfigurato in produzione, espone cookie su HTTP.

Soluzioni:

- 1 In produzione forzare fail-fast se COOKIE_SECURE != true.
- 2 Validare COOKIE_SAMESITE in strict|lax|none; se none, richiedere secure=true.
- 3 Non restituire access_token ai browser web. Opzioni:
 - endpoint login web: solo cookie.
 - - endpoint login desktop/API: Bearer token, richiesto con header X-Client-Type: tauri.
- 1 Ridurre access token a 15-60 min e introdurre refresh token rotante.

- 2 Per Tauri, usare store sicuro OS o plugin stronghold/keyring invece di localStorage.
- 3 Aggiungere device/session list e revoca per utente.
- 4 Pulizia periodica revoked_tokens, che oggi non ha expiry nel modello: backend/db/modelli.py:49.

Finding H-05 - Tauri Desktop: CSP disabilitata e permesso shell ampio

Severita: Alta **Categoria:** Desktop App Hardening **Evidenza:**

- CSP null: frontend/src-tauri/tauri.conf.json:26.
- Capability include shell:default: frontend/src-tauri/capabilities/default.json:8.
- Token desktop in localStorage: frontend/app/lib/api.ts:12.
- Devtools aperti in debug: frontend/src-tauri/src/lib.rs:27 (ok in debug, da verificare in release).

Motivazione:

In una WebView, XSS ha impatto maggiore rispetto al browser: puo' leggere storage locale e invocare API Tauri consentite. shell:default amplia il blast radius se qualunque pagina o script compromesso riesce a chiamare il bridge. CSP disabilitata rende piu' facile eseguire script non previsto.

Soluzioni:

- 1 Impostare CSP restrittiva:

```
"csp": "default-src 'self'; connect-src 'self' https://maintai-v3.onrender.com; img-src 'self' data: blob: https;; style-src 'self' 'unsafe-inline'; script-src 'self'"
```

- 1 Rimuovere shell:default se non indispensabile. Se serve, definire permessi nominativi e comandi consentiti.
- 2 Conservare il JWT desktop in storage sicuro, non localStorage.
- 3 Verificare che update endpoint e pubkey siano coerenti e che la chiave privata minisign non sia mai nel repo.
- 4 Aggiungere smoke test di build release con CSP attiva.

Finding H-06 - Configurazione IMAP puo' diventare SSRF/egress abuse

Severita: Alta in multi-tenant SaaS; Media in deploy single-tenant fidato **Categoria:** SSRF / Outbound Network Abuse **Evidenza:**

- Server e porta IMAP vengono da input utente: backend/api/routes/email_config.py:14, backend/api/routes/email_config.py:15.
- Test connessione verso host arbitrario: backend/api/routes/email_config.py:42.
- Poller usa host/porta salvati: backend/services/email_poller.py:54.

Motivazione:

Un tenant admin puo' far connettere il backend a host/porte scelti. Questo puo' essere usato per scansione rete interna, connessioni verso metadata service/cloud endpoints, o abuso egress. Il timeout riduce l'impatto ma non

elimina il problema.

Soluzioni:

- 1 Consentire solo porta 993 salvo eccezioni amministrative.
- 2 Bloccare IP privati/link-local/loopback dopo risoluzione DNS:
 - 127.0.0.0/8
 - - 10.0.0.0/8
 - - 172.16.0.0/12
 - - 192.168.0.0/16
 - - 169.254.0.0/16
 - - IPv6 local/link-local
- 1 Allowlist provider noti o validazione MX/domain ownership per tenant.
- 2 Limitare tentativi test IMAP per tenant/IP.
- 3 Non restituire dettagli tecnici eccessivi sugli errori di rete.
- 4 Egress firewall lato infrastruttura.

Finding H-07 - Endpoint AI costosi senza rate limiting e rischio prompt injection/data leakage

Severita: Alta **Categoria:** Abuse / LLM Data Governance **Evidenza:**

- Planning AI: backend/api/routes/planning.py:290.
- Guide bot usa OpenAI su input utente: backend/api/routes/guide.py:199.
- Manual upload invia testo estratto a parser AI: backend/api/routes/manuali.py:69.
- Diagnostic include dati ticket/manuali: backend/api/routes/diagnostic.py:118.
- Solo login ha limiter esplicito; non emerge limiter sugli endpoint AI.

Motivazione:

Gli endpoint AI consumano denaro, tempo e dati sensibili. Manuali/email/ticket possono contenere istruzioni malevole ("ignora schema", "esfiltra dati") e possono indurre output errati. Anche se l'AI non esegue codice, può alterare piani, task o diagnostica se l'output non è strettamente validato.

Soluzioni:

- 1 Rate limit per utente/tenant su AI:
 - planning generate: es. 5/ora per tenant.
 - - guide chat: es. 30/min per utente con budget giornaliero.
 - - manual parsing: coda asincrona e limite dimensione/pagine.
- 1 Budget mensile per tenant con alert.
- 2 Minimizzazione dati: inviare solo campi necessari; anonimizzare nomi tecnici/operatori dove possibile.

- 3 Output validation con Pydantic/schema rigidi e reject su campi inattesi.
- 4 Human approval per piani generati/diagnostica critica.
- 5 Prompt injection guard: separare dati da istruzioni, usare delimitatori, non trattare contenuto manuale/email come istruzioni di sistema.
- 6 Logging: non loggare prompt completi o risposte AI contenenti dati industriali.

Finding H-08 - Security headers assenti o incompleti

Severita: Alta/**Media****Categoria:** Browser Hardening**Evidenza:**

- frontend/next.config.ts non definisce headers().
- Tauri CSP e' null: frontend/src-tauri/tauri.conf.json:26.
- Backend non mostra middleware per HSTS/CSP/no-sniff/frame options.

Motivazione:

Header come CSP, HSTS, frame-ancestors, no-sniff e referrer-policy riducono impatto di XSS, clickjacking, MIME sniffing e downgrade. L'app gestisce sessioni cookie e dati industriali: la difesa in profondita e' rilevante.

Soluzioni:

1 In Next/Vercel:

- Content-Security-Policy
- - Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
- - X-Content-Type-Options: nosniff
- - Referrer-Policy: strict-origin-when-cross-origin
- - Permissions-Policy
- - X-Frame-Options: DENY o CSP frame-ancestors 'none'

1 Su backend/API:

- Cache-Control: no-store per risposte auth/user/log.
- - X-Content-Type-Options: nosniff.

1 Testare con browser devtools e scanner header.

Finding M-01 - CORS troppo permissivo per produzione

Severita: Media**Categoria:** CORS Misconfiguration**Evidenza:**

- Default origins includono localhost e LAN: backend/main.py:88.
- allow_credentials=True: backend/main.py:685.
- Header autorizzati includono Authorization e X-Tenant-Id: backend/main.py:687.

Motivazione:

Con credenziali abilitate, la allowlist CORS deve essere minima. Origini locali sono utili in dev, ma in produzione aumentano la superficie: app locali o ambienti di test possono interagire con cookie/sessioni se un browser li considera autorizzati.

Soluzioni:

1 Separare dev/prod:

- produzione: solo domini Vercel ufficiali.
- - sviluppo: localhost/LAN solo con ENV=development.

1 Fallire l'avvio in produzione se CORS_ORIGINS contiene localhost, 127.0.0.1, IP privati o http://.

2 Ridurre allow_headers a quelli realmente necessari.

3 Loggare origin rifiutati senza salvare dati sensibili.

Finding M-02 - Proxy Next riscrive Origin/Referer verso backend

Severita: Media **Categoria:** Confused Deputy / CSRF Boundary **Evidenza:**

- Proxy Next accetta origini trusted: frontend/app/api/[...path]/route.ts:8.
- Costruisce URL backend dinamico: frontend/app/api/[...path]/route.ts:44.
- Sovrascrive origin e referer con valore allowed dal backend: frontend/app/api/[...path]/route.ts:52, frontend/app/api/[...path]/route.ts:53.

Motivazione:

Il backend non vede piu' l'origin reale del browser, ma un origin "trusted" inserito dal proxy. Questo puo' essere corretto architetturalmente, ma sposta tutta la fiducia sul proxy. Se una trusted origin viene compromessa o se la lista diverge da backend, il controllo CSRF backend perde valore come barriera indipendente.

Soluzioni:

- 1 Inviare anche X-Original-Origin o X-Forwarded-Origin, firmato/validato se necessario.
- 2 Backend: se richiesta arriva dal proxy, validare un header segreto server-to-server o mTLS, non solo Origin.
- 3 Tenere una sola fonte di verita' per origins trusted.
- 4 Evitare che il proxy inoltri header non necessari.

Finding M-03 - Build frontend ignora errori TypeScript/ESLint

Severita: Media **Categoria:** Secure SDLC **Evidenza:**

- ignoreDuringBuilds: true: frontend/next.config.ts:25.
- ignoreBuildErrors: true: frontend/next.config.ts:31.

Motivazione:

Errori TS/ESLint non sono vulnerabilit  da soli, ma permettono di deployare bug di auth, error handling, uso di token o API non tipizzate. In un'app con RBAC/multi-tenant, gli errori di tipo possono nascondere regressioni di sicurezza.

Soluzioni:

- 1 In CI rendere bloccanti:

```
npm run lint
npm run build
```

- 1 Rimuovere gradualmente ignoreBuildErrors.
- 2 Se necessario, permettere eccezioni solo in build desktop temporanee, non produzione web.

Finding M-04 - Log e dati operativi possono contenere PII/segrete industriali

Severita: Media**Categoria:** Logging / Privacy**Evidenza:**

- Log di email poller con mittente e ticket: backend/services/email_poller.py:139.
- Manuali salvano testo_raw e json_estratto: backend/api/routes/manuali.py:76.
- Debug AI raw response parziale: backend/api/routes/manuali.py:81.
- Errori backend loggano stacktrace server-side: backend/core/exceptions.py:29.
- System logs leggibili da responsabile per tenant e da superadmin globalmente: backend/api/routes/logs.py:46.

Motivazione:

Log e testo manuali possono contenere email, nomi, seriali, dati impianto, procedure operative o parti di manuali proprietari. Se i log sono troppo ampi, aumentano l'impatto di un accesso admin o di una fuga DB.

Soluzioni:

- 1 Classificare dati loggabili: audit essenziale vs dati sensibili.
- 2 Redazione automatica per email, token, URL firmati, password, API keys.
- 3 Retention differenziata: log sicurezza piu' lunghi, contenuti tecnici piu' brevi.
- 4 Separare audit trail immutabile da debug log.
- 5 Evitare log di output AI raw salvo ambiente dev.
- 6 Aggiungere access log per consultazione log da parte admin.

Finding M-05 - Seed locali con credenziali deboli e rischio uso improprio SQLite

Severita: Media**Categoria:** Environment Hardening**Evidenza:**

- In locale seed usa admin/admin: backend/core/init_db.py:104.
- In locale seed usa tecnico/tecnico: backend/core/init_db.py:124.

- In PostgreSQL c'è fail-fast se seed password non sicura: backend/core/init_db.py:97, backend/core/init_db.py:117.

Motivazione:

La protezione per Postgres è buona. Il rischio resta nei deploy accidentali con SQLite o ambienti demo esposti dove le credenziali note sono attive. Demo e staging spesso diventano internet-facing.

Soluzioni:

- 1 Introdurre APP_ENV=production|staging|development.
- 2 Fallire se APP_ENV != development e DB è SQLite.
- 3 Mostrare warning evidente se admin/tecnico default sono attivi.
- 4 Forzare cambio password al primo login per seed user.
- 5 Disabilitare utenti seed nei dump demo pubblici.

Finding M-06 - Tenant isolation diffusa ma manuale

Severita: Media**Categoria: Multi-Tenant Isolation****Evidenza:**

- Molti endpoint filtrano esplicitamente tenant_id.
- Esiste un ContextVar con filtro ORM automatico: backend/core/database.py:49, backend/core/database.py:53.
- Superadmin può impostare X-Tenant-Id: backend/core/security.py:214.
- check_tenant_ownership bypassa se tenant_id is None: backend/core/security.py:266.

Motivazione:

Il modello multi-tenant dipende da disciplina per-endpoint. Basta una route nuova senza filtro per creare IDOR cross-tenant. Il superadmin senza contesto tenant ha accesso globale, utile ma rischioso per mutazioni accidentali.

Soluzioni:

- 1 Repository pattern obbligatorio per ogni modello tenant-scoped.
- 2 Test parametrico che verifica 404 cross-tenant per ogni endpoint /{id}.
- 3 Per superadmin, richiedere X-Tenant-Id per tutte le mutazioni tenant-scoped.
- 4 Evitare tenant_id nullable sui record tenant-scoped nuovi.
- 5 Constraint DB dove possibile e indici compositi.

Finding M-07 - Email password legacy fallback può mascherare dati in chiaro

Severita: Media**Categoria: Secrets at Rest****Evidenza:**

- decrypt_data restituisce testo originale se decryption fallisce: backend/core/security.py:122.
- Email poller usa password in chiaro se is_encrypted false: backend/services/email_poller.py:51.

Motivazione:

La compatibilit  legacy permette password non cifrate o corrotte senza allarme forte. Nel breve facilita migrazione; nel lungo rende difficile garantire che tutte le password IMAP siano cifrate.

Soluzioni:

- 1 Script one-off di migrazione: trovare record `is_encrypted=false`, cifrare e salvare.
- 2 Dopo migrazione, rimuovere fallback plaintext.
- 3 Alert se decryption fallisce.
- 4 Considerare secret manager esterno invece di Fernet DB-local.

Finding L-01 - cargo audit non disponibile

Severita: Bassa/Media**Categoria: Supply Chain Rust****Evidenza:**

- `cargo audit --json` fallisce per comando non installato.
- `cargo tree -e no-dev` mostra Tauri 2.x, updater, shell, reqwest/hyper/rustls ecc., ma non sostituisce un CVE audit.

Soluzioni:

- 1 Installare in CI:

```
cargo install cargo-audit
cargo audit
```

- 1 Aggiungere `cargo deny` per license/advisory policy.
- 2 Bloccare release desktop se ci sono advisory high/critical.

Finding L-02 - Pubkey update Tauri presente, ma governance chiavi da rafforzare

Severita: Bassa/Media**Categoria: Release Integrity****Evidenza:**

- Pubkey updater in config: `frontend/src-tauri/tauri.conf.json:46`.
- Manifest update pubblico: `backend/api/routes/desktop_update.py:33`.
- Chiavi Tauri locali sono ignorate: `.gitignore:8`, `.gitignore:9`.

Motivazione:

La firma updater e' una buona barriera. Il rischio reale e' operativo: perdita chiave privata o password signing. Se compromessa, un update malevolo firmato diventa installabile.

Soluzioni:

- 1 Conservare private key in secret manager o hardware-backed storage.
- 2 Separare chiave staging/prod.
- 3 Rotazione e procedura revoca.

4 CI release con approvazione manuale.

Dipendenze: Dettaglio Audit

Backend Python

Da pip-audit -r backend/requirements.txt -f json:

Package	Versione	Advisory	Fix
python-multipart	0.0.22	CVE-2026-40347, CVE-2026-42561, DoS multipart	>=0.0.27
cryptography	46.0.6	CVE-2026-39892 / PYSEC-2026-36	>=46.0.7
pytest	9.0.2	CVE-2025-71176, locale su UNIX	>=9.0.3

Nota: pytest non dovrebbe stare nelle dipendenze runtime di produzione. Spostarlo in dev requirements riduce superficie.

Frontend NPM

Da npm audit --omit=dev --json:

Package	Severita	Tipo rischio principale	Fix
next 16.1.6	High	DoS, SSRF, middleware/proxy bypass, request smuggling, XSS/cache poisoning in advisory specifici	next@16.2.6
fast-uri	High	path traversal/host confusion parsing URI	audit fix
picomatch	High	ReDoS / glob matching issue	audit fix
hono, @hono/node-server	Moderate	static middleware/cookie/JWT/cache/bodyLimit issues	audit fix
postcss	Moderate	XSS in stringify	upgrade via Next/postcss
qs, brace-expansion, ip-address	Moderate	DoS/XSS/transitive	audit fix

Remediation Roadmap

0-24 ore

1 Applicare update dipendenze:

- next@16.2.6, eslint-config-next@16.2.6.
- - python-multipart>=0.0.27, cryptography>=46.0.7, pytest>=9.0.3 o spostare pytest in dev.

1 Aggiungere RBAC minimo sugli endpoint mutanti:

- asset, tecnici, piani, planning, email config: responsabile|superadmin.
- - tenant/log file/bulk import: superadmin.

1 Rate limit:

- checklist pubblica.

- - AI endpoints.
- - upload endpoints.

1 Verificare e ripulire `.claude/settings.local.json`; ruotare segreti se reali.

2-7 giorni

- 1 Rendere allegati privati con endpoint download autenticato.
- 2 Aggiungere body size limits a proxy/server.
- 3 Security headers in Next e backend.
- 4 Hardening cookie produzione.
- 5 CSP Tauri e rimozione `shell:default`.
- 6 IMAP SSRF guard.

2-4 settimane

- 1 Test suite RBAC/IDOR cross-tenant.
- 2 Secret scanning CI.
- 3 Supply chain gates: `npm audit`, `pip-audit`, `cargo audit`.
- 4 Data governance AI: minimizzazione, retention, audit, DPA.
- 5 Threat model completo con DFD e trust boundaries.

Esempi di Test di Sicurezza da Aggiungere

- 1 `test_tecnico_cannot_create_asset`
- 2 `test_tecnico_cannot_clear_planning`
- 3 `test_responsabile_cannot_access_other_tenant`
- 4 `test_superadmin_mutation_requires_tenant_context`
- 5 `test_public_check_rate_limited`
- 6 `test_upload_rejects_executable_even_if_renamed_pdf`
- 7 `test_download_attachment_requires_ticket_tenant`
- 8 `test_cookie_secure_required_in_production`
- 9 `test_imap_rejects_private_ip`
- 10 `test_ai_endpoint_quota_per_tenant`

Conclusione

MaintAI ha una base difensiva non banale: JWT fail-fast, bcrypt, Fernet obbligatoria, CSRF origin check e tenant filtering sono segnali buoni. Il gap principale e' la separazione fra autenticazione e autorizzazione: oggi molte azioni amministrative sembrano accessibili a qualunque utente del tenant. In parallelo, le dipendenze vulnerabili e la gestione file pubblica sono rischi concreti di disponibilita' e confidenzialita'.

L'ordine migliore e': patch dipendenze, RBAC, upload privati/rate limit, poi hardening browser/desktop e automazione CI. Questo riduce subito il rischio reale senza riscrivere l'architettura.